

MZmine3 Security Review

Date: 2026-03-24 **Scope:** Full codebase **Reviewer:** Claude (automated security analysis)

Summary

Two confirmed vulnerabilities were found. No findings related to the recent `mzconnect` branch changes — those are clean.

#	Title	Severity	File(s)	Confidence
1	XXE in XML parsers	HIGH	XMLUtils.java, MzXMLImportTask.java, MzDataImportTask.java, + more	8/10
2	AES/ECB credential encryption	MEDIUM	StringCrypter.java	9/10

Finding 1 — XXE in XML Parsing

Severity: HIGH **Category:** xxe **Confidence:** 8/10

Affected Files

- `mzmine-community/src/main/java/io/github/mzmine/util/XMLUtils.java` (lines 65-84)
- `...modules/io/import_rawdata_mzxml/MzXMLImportTask.java` (lines 143-150)
- `...modules/io/import_rawdata_mzdata/MzDataImportTask.java` (lines 151-156)
- `...modules/io/projectload/version_3_0/RawDataFileOpenHandler_3_0.java` (line 147)
- `...main/impl/MZmineConfigurationImpl.java` (lines 254, 340)
- `...modules/batchmode/BatchComponentController.java` (line 387)
- Additional project and parameter loading files

What is the problem?

Every `DocumentBuilderFactory.newInstance()` and `SAXParserFactory.newInstance()` call in the codebase creates parsers with **default settings**, which in Java allow processing of external entities and DTDs. A codebase-wide search found **zero** instances of any XXE mitigation. Java does not disable XXE by default (verified against the project's Java 25 target).

Attack Scenario

An attacker crafts a malicious `mzXML` or `mzData` file containing:

```
<?xml version="1.0"?>
<!DOCTYPE foo [
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<mzXML>&xxe;</mzXML>
```

When a user opens this file in MZmine, the XML parser fetches the external resource and embeds its contents. This can:

- **Exfiltrate local files** (e.g., SSH keys, config files, research data)
- **Probe internal network services** via http:// references

Since mzXML and mzData are standard scientific data exchange formats, a user receiving data from a collaborator or downloading from a public repository could unknowingly open a malicious file.

Fix

Create a single hardened factory helper and use it everywhere XML is parsed. Add a static `XMLUtils.newSecureDocumentBuilderFactory()` method and replace all unguarded usages:

```
// Secure DocumentBuilderFactory
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
factory.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);
factory.setFeature("http://xml.org/sax/features/external-general-entities", false);
factory.setFeature("http://xml.org/sax/features/external-parameter-entities",
false);
factory.setFeature("http://apache.org/xml/features/nonvalidating/load-external-
dtd", false);
factory.setXIncludeAware(false);
factory.setExpandEntityReferences(false);
```

Finding 2 — AES/ECB Used for Credential Encryption

Severity: MEDIUM **Category:** weak_crypto **Confidence:** 9/10

Affected file: mzmne-

community/src/main/java/io/github/mzmne/util/StringCrypter.java (line 99)

What is the problem?

`Cipher.getInstance("AES")` defaults to **AES/ECB/PKCS5Padding** in Java. ECB mode is deterministic — identical plaintext blocks always produce identical ciphertext blocks, leaking patterns. No IV is used and there is no authentication (no GCM/HMAC).

This cipher encrypts real user credentials:

- GNPS database username/password (used for submitting research data)
- Email server password (used for error reporting)

The AES encryption key (`EncryptionKeyParameter`) is stored in the **same config file** as the encrypted passwords (`~/.mzmine/conf.xml`). The application's own loading code acknowledges this dependency (comment at `MZmineConfigurationImpl.java` lines 271-274).

Attack Scenario

Any process with read access to `~/.mzmine/conf.xml` obtains both the base64-encoded key and all encrypted credentials in one file. Decrypting them takes a few lines of code:

```
from Crypto.Cipher import AES
import base64

key = base64.b64decode("<key from conf.xml>")
ct = base64.b64decode("<password from conf.xml>")
AES.new(key, AES.MODE_ECB).decrypt(ct) # plaintext password
```

Fix

1. `Cipher.getInstance("AES")` with `Cipher.getInstance("AES/GCM/NoPadding")` using a random 12-byte IV prepended to the ciphertext.
2. Store the encryption key in the OS credential store instead of the config file:
 - macOS: Keychain via `java.security.KeyStore` with `KeychainStore` provider
 - Windows: DPAPI / Windows-MY `KeyStore`
 - Linux: Secret Service / `~/.gnome-keyring`

What Was Reviewed But Found Clean

- `mzconnect` branch changes (new `MzConnectAnalogCompoundDatabaseMatchesType`, context menu update, annotation priority list) — no security issues
- `ZipUtils.unzipStream` — vulnerable pattern present but the method is dead code (zero callers)
- `NistMsSearchTaskRuntime.exec()` usage — not exploitable for command injection; Java's `exec(String)` does not invoke a shell
- No hardcoded API keys or tokens found
- No unsafe deserialization (`ObjectInputStream`) patterns found in application code

Recommended Next Steps

3. **Short term:** Add XXE mitigations to `XMLUtils.java` and all direct `DocumentBuilderFactory` / `SAXParserFactory` usages. This is the higher-severity issue and has a straightforward, well-understood fix.
4. **Medium term:** Migrate `StringCrypter` to AES/GCM and explore OS keystore integration for the encryption key.

5. **Housekeeping:** Remove or fix `ZipUtils.unzipStream` to prevent future misuse if a caller is added.